

知識グラフからの Horn 規則マイニング

林 克彦^{*1}

東京大学大学院総合文化研究科
言語情報科学専攻

katsuhiko-hayashi@g.ecc.u-tokyo.ac.jp

1 事前準備

1.1 知識グラフ (Knowledge Graph)

ここでは RDF に基づく知識グラフ (KG) に焦点を当てる. KG は事実 (fact) の集合と考えることができ, fact は (s, r, o) の triple 形式で表される. s を主語 (subject), r を関係 (relation または predicate), o を目的語 (object) と呼ぶ. また, 論理的な記法として, fact を $r(s, o)$ と表記する. さらに, relation r の逆関係 (inverse relation) を r^{-1} として定義する. r^{-1} は $r(s, o) \in \mathcal{K}$ に対して, $r^{-1}(o, s)$ という形式の fact を形成する. 図1は KG の例であり, $characterIn(Spock, StarTrek)$ は fact の例である.

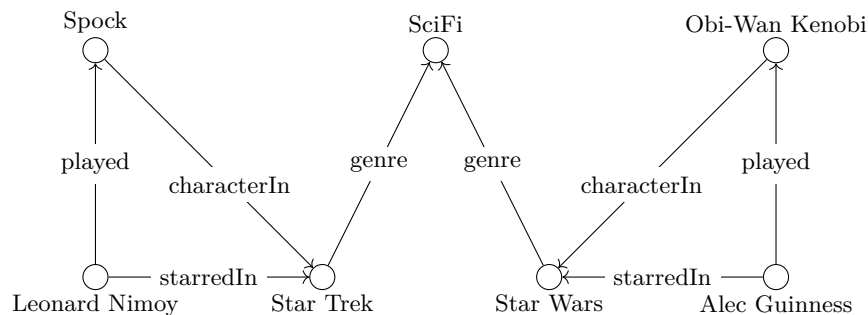


図1 KG の例 (文献 [3] から引用).

KG $\mathcal{K}$ に対して, $\mathcal{R} = \pi_{\text{rel}}(\mathcal{K})$ を $\mathcal{K}$ に含まれる relation の集合, $\mathcal{E} = \pi_{\text{subj}}(\mathcal{K}) \cup \pi_{\text{obj}}(\mathcal{K})$ を $\mathcal{K}$ に含まれる **個物** (entity) の集合とする. 図1の KG を $\mathcal{K}$ とすると, relation の集合は $\mathcal{R} = \{characterIn, genre, played, starredIn\}$, entity の集合は $\mathcal{E} = \{AlecGuinness, LeonardNimoy, Obi-WanKenobi, SciFi, Spock, StarTrek, StarWars\}$ となる.

^{*1} 著者個人が CC BY-SA 4.0 に従って公開を行っています. ライセンスに従っている限り, この資料の使用は自由ですが, 使用に伴い生じた損害には対応できません. 資料の内容に関する問い合わせはメールでお願いします.

1.2 アトムと Horn 規則 (Atoms and Rules)

subject や object に変数を取りうる fact のことを atom と呼ぶ。atom が**インスタンス化**されている (instantiated) と、subject か object のうち少なくとも 1 つは定数となる。例えば、 $livesIn(x, Berlin)$ はインスタンス化された atom となる (ここで x は変数、 $Berlin$ は定数)。subject と object の両方が定数となる場合、その atom は**グラウンド化**されている (grounded) と呼び、fact と等価である。置換 (substitution) σ とは変数から定数への部分写像であり、atom やその**論理積** (conjunction) $\wedge$ に対して自明に拡張できる。substitution の例は節2で触れる。

Horn 規則 (rule) は**ヘッド** (head) と**ボディ** (body) から構成され、head は 1 つの atom、body は atom の集合から構成される。atom $r(x, y)$ を head、atom の集合 $\{B_1, B_2, \dots, B_n\}$ を body とする rule を**論理包含** (implication) $\Rightarrow$ によって以下のように表す。

$$B_1 \wedge B_2 \wedge \dots \wedge B_n \Rightarrow r(x, y)$$

rule の記法を省略する場合、 $\vec{B} \Rightarrow r(x, y)$ と表記する。2 つの Atom A, A' が変数または entity (定数) を共有するとき、それらは**連結** (connected) であるという。もし、rule に出現する各 atom が他のどの atom に対しても推移的に connected である (いくつかの atom を経由して connected な関係となる) 場合、その rule は connected であると言う。

rule に現れる全ての変数が少なくとも 2 つの Atom に現れるとき、その rule は**閉じている** (closed) と言う。closed な rule は常に**安全** (safe) と呼ばれるが、これは rule の head に現れる全ての変数は body にある少なくとも 1 つの atom にも現れることが保証されるからである。

1.3 関係と関数 (Relations and Functions)

relation があらゆる subject に対して、たかだが 1 つの object しか取らないとき、その relation を**関数** (function) と呼ぶ。すなわち、 $\forall s : |o : r(s, o) \in \mathcal{K}| \leq 1$ となる relation r が function である。**逆関数** (inverse function) とは各 object がたかだが 1 つの subject しか持たない function のことである。

一般に KG は多くのノイズを含むため、 $hasBirthdate$ のような function となる relation であっても同じ subject に対して複数の object を持つことが起こりうる。このような問題に対処するため、**関数性** (functionality) という概念を用いる。relation r の functionality とは 0 から 1 の間の値で表され、1 のときに r は function となる次のような指標である。

$$fun(r) := \frac{|s : \exists o : r(s, o) \in \mathcal{K}|}{|(s, o) : r(s, o) \in \mathcal{K}|}.$$

逆関数性 (inverse functionality) を $ifun(r) := fun(r^{-1})$ として定義する。一般的に、Web データから半自動的に構築された KG では $\forall r \in \mathcal{R} : fun(r) \geq ifun(r)$ となることが人手により検証されている。rule マイニングではこの条件を満たさない関係は inverse relation に変換して処理される。

2 Horn 規則マイニング

rule $R = B_1 \wedge B_2 \wedge \dots \wedge B_n \Rightarrow H$ と substitution σ が与えられたとき、 $\sigma(R)$ を rule の**インスタンス化** (instantiation) と呼ぶ。また、 $\forall i \in \{1, \dots, n\} : \sigma(B_i) \in \mathcal{K}$ であるとき、 $\sigma(H)$ を $\mathcal{K}$ から R の予

表1 KG の例 (表形式): 2つの relation *livesIn* と *wasBornIn* と 5つの fact から構成される.

<i>livesIn</i>	<i>wasBornIn</i>
(Adam, Paris)	(Adam, Paris)
(Adam, Rome)	(Carl, Rome)
(Bob, Zurich)	

測 (prediction) と呼ぶ. prediction は $\mathcal{K} \wedge R \models \sigma(H)$ と書く. 表1の KG を例として, rule $livesIn(x, y) \Rightarrow wasBornIn(x, y)$ を考える. このとき, substitution $\sigma = \{x \rightarrow Adam, y \rightarrow Paris\}$ とすると, prediction は $wasBornIn(Adam, Paris)$ となる.

一般に, RDF KG には fact の欠損が含まれる. 図2に示すように, 不完全な KG における Fact は 4 種類に大別される:

- KG True: KG で既知 (Known) の true な fact
- New True: 未知 (Unknown) の true な fact
- KG False: KG で既知 (Known) の false な fact
- New False: 未知 (Unknown) の false な fact

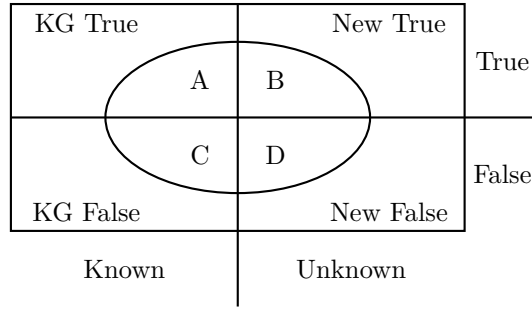


図2 KG が不完全である場合の Prediction (文献 [2] から引用).

rule は図2の円で表した prediction を作る. B の fact をできるだけ多く prediction し, D の fact をできるだけ少なく prediction するような rule をマイニングすることが理想的である. しかし, New True と New False の領域は未知であり, RDF KG では KG False の領域も空であることが一般的である. そのため, rule マイニングではこのような状況に応じて rule の良さを評価する尺度が検討されてきた.

2.1 支持度 (Support)

rule R の良さを評価する尺度の 1 つに**支持度** (support) がある. support には様々な定義が考えられるが, ここでは文献 [2] に従って次のように定義する.

$$support(R) := |p : (\mathcal{K} \wedge R \models p) \wedge p \in \mathcal{K}|$$

support は図2における KG True の領域をどれだけ prediction できるか評価する尺度となっている. 表1を例として考えると, rule $livesIn(x, y) \Rightarrow wasBornIn(x, y)$ の prediction は $wasBornIn(Adam, Paris)$ のみ KG True の領域にあるので $support(livesIn(x, y) \Rightarrow wasBornIn(x, y)) = 1$ となる.

support を比率として考える尺度として**ヘッド被覆率** (head-coverage) がある.

$$hc(\vec{B} \Rightarrow r(x, y)) := \frac{\text{support}(\vec{B} \Rightarrow r(x, y))}{|(x, y) : r(x, y) \in \mathcal{K}|} = \frac{\text{support}(\vec{B} \Rightarrow r(x, y))}{\text{size}(r(x, y))}$$

ここで $\text{size}(r(x, y)) = |(x, y) : r(x, y) \in \mathcal{K}|$ である. 表1を例とすると, $\text{livesIn}(x, y) \Rightarrow \text{wasBornIn}(x, y)$ の head-coverage は $1/2$ となる. support と head-coverage には rule の body に新しい atom が加えられる度, その値が単調に減少していくという性質がある.

2.2 確信度 (Confidence)

KG False の領域を考えることは rule の prediction に対する反例として有用である. このような負例を利用した評価尺度に**確信度** (confidence) がある.

$$\text{conf}(R) := \frac{\text{support}(R)}{\text{support}(R) + |p : (\mathcal{K} \wedge R \models p) \wedge p \in \text{cex}(R)|}$$

ここで $\text{cex}(R)$ は rule R に対する負例の集合である. すなわち, confidence とは rule によって作り出される true と false となる prediction において, true となる prediction がどの程度の割合となるかを表す尺度となっている.

しかし, 前述したように KG では KG False に関する情報は与えられていないことが一般的である. そのため, 負例を定義するには KG に対して仮説が必要となる. 代表的な仮説としては次の 2 つが知られている.

- **閉世界仮説** (CWA; Closed World Assumption): KG に存在しない全ての fact は負例と考える
- **部分完全性仮説** (PCA; Partial Completeness Assumption): $r(s, o) \in \mathcal{K}$ に対して, $\text{fun}(r) \geq \text{ifun}(r)$ であるとき, fact $r(s, o') \notin \mathcal{K}$ は全て負例と考える.

CWA に基づくと, KG に登録されていない事実は全て負例として扱われるため, $\text{conf}(R)$ は次のように再定義できる.

$$\text{conf}(R) = \frac{\text{support}(R)}{\text{support}(R) + |p : (\mathcal{K} \wedge R \models p) \wedge p \notin \mathcal{K}|}$$

表1を例とすると, $\text{livesIn}(x, y) \Rightarrow \text{wasBornIn}(x, y)$ の CWA に基づく confidence は $1/3$ となる. これは分母の負例として, 2 つの fact $\text{wasBornIn}(\text{Adam}, \text{Rome})$ と $\text{wasBornIn}(\text{Bob}, \text{Zurich})$ が考えられるためである (これらはどちらも KG には存在しない).

PCA は文献 [2] で提案された仮説であり, PCA に基づいて定義された $\text{cex}(R)$ を用いたときの confidence を $\text{conf}_{\text{pca}}(R)$ とする. ここで $R = \vec{B} \Rightarrow r(x, y)$ としたとき,

$$\begin{aligned} \text{conf}_{\text{pca}}(R) &= \frac{\text{support}(R)}{\text{support}(R) + |(x, y) : (\mathcal{K} \wedge R \models r(x, y)) \wedge r(x, y') \in \mathcal{K} \wedge r(x, y) \notin \mathcal{K}|} \\ &= \frac{\text{support}(R)}{|(x, y) : (\mathcal{K} \wedge R \models r(x, y)) \wedge r(x, y') \in \mathcal{K}|} \end{aligned}$$

と定義される. 表1の例を考えると, $\text{livesIn}(x, y) \Rightarrow \text{wasBornIn}(x, y)$ の PCA に基づく confidence は $1/2$ となる. wasBornIn は 1 つの subject に対して, 1 つの object をとるため, PCA に基づいて, $\text{wasBornIn}(\text{Adam}, \text{Rome})$ は負例と考えられる ($\text{wasBornIn}(\text{Adam}, \text{Paris})$ が KG に存在するため). 一方, KG には wasBornIn に Bob を subject とする fact は存在しないため, $\text{wasBornIn}(\text{Bob}, \text{Zurich})$ は未知として扱われる.

Algorithm 1 Horn 規則マイニング

```
1: function AMIE(KG  $\mathcal{K}$ ,  $minHC$ ,  $maxLen$ ,  $minConf$ )
2:    $q := [r_1(x, y), r_2(x, y), \dots, r_m(x, y)]$ 
3:    $out := \langle \rangle$ 
4:   while  $q$  is not empty do
5:      $r = q.dequeue()$ 
6:     if AcceptedForOutput( $r, out, minConf$ ) then
7:        $out.add(r)$ 
8:     if  $length(r) < maxLen$  then
9:        $R(r) = \text{Refine}(r)$ 
10:    for all rules  $r_c \in R(r)$  do
11:      if  $hc(r_c) \geq minHC \wedge r_c \notin q$  then
12:         $q.enqueue(r_c)$ 
13:  return  $out$ 
```

2.3 AMIE アルゴリズム

Horn 規則マイニングの目的は評価尺度の値が閾値以上の rule を発見することである。これを効率的に探索するため、文献 [2, 1] では **AMIE** (Association Rule Mining under Incomplete Evidence) アルゴリズムが提案されている。

AMIE アルゴリズムの概略を Algorithm 1 に示す。入力として KB $\mathcal{K}$, $minHC$, $maxLen$, $minConf$ を受け取る。 $minHC$ はマイニングする rule の head-coverage に関する閾値, $maxLen$ は長さに関する閾値, $minConf$ は confidence に対する閾値を表す。2 行目では head だけからなる長さ 1 の rule を Queue q に入れる。Queue から rule を逐次的に dequeue しながら処理は進められる。6-7 行目では条件を満たす rule が出力となる。8 行目では rule の長さが $maxLen$ を越えていなければ、後述する Refinement 処理によって、その rule が拡張され、新しい rule が作り出される。これらの新しい rule は 10 行目で head-coverage が閾値を上回り、同じ rule が Queue に存在しなければ、新しい候補として Queue に追加される。

Refinement の処理として文献 [2] では以下の 3 種類の演算が提案されている。

- **Add Dangling Atom** ($\mathcal{O}_D$): この演算では新しい atom を rule に追加する。atom は subject か object の一方に新しい変数を持ち、もう一方は rule 中の atom に含まれる変数を持つ。
- **Add Instantiated Atom** ($\mathcal{O}_I$): この演算では新しい atom を rule に追加する。atom は subject か object の一方に entity (定数) を持ち、もう一方は rule 中の atom に含まれる変数を持つ。
- **Add Closing Atom** ($\mathcal{O}_C$): この演算では新しい atom を rule に追加する。atom は subject と object の両方で rule 中の atom に含まれる要素を持つ。

上記の演算は全て connected な rule を作り出すことが保証されている。一方、closed な rule であることは保証されない。この演算の詳細は節 2.4 で説明する。

AMIE アルゴリズムで dequeue された全ての rule が出力とはならない。これは rule が closed ではない、あるいは、既に出力された rule の方が有望な場合があるためであり、Algorithm 2 ではこのような観点から

Algorithm 2 rule 出力の決定

```
1: function ACCEPTEDFOROUTPUT(rule  $r$ ,  $out$ ,  $minConf$ )
2:   if  $r$  is not closed  $\vee conf_{pca}(r) < minConf$  then
3:     return false
4:    $parents = parentsOfRule(r, out)$ 
5:   for all  $r_p \in parents$  do
6:     if  $conf_{pca}(r) \leq conf_{pca}(r_p)$  then
7:       return false
8:   return true
```

rule を出力するかしないかをチェックする. 2 行目では最初に rule が closed であるかをチェックし, 次に, rule の confidence が閾値以上となるかをチェックする. また, 6 行目では rule の親となる rule と比較して, confidence の向上が見られるかをチェックする. rule の親とはその rule と同じ head, 及び, その rule の body にある atom の subset から body が構成され, 既に出力された (out に追加済み) rule のことである. support や head-coverage は単調な尺度であるため, ある rule がその親より高いスコアを持つことはない. もし confidence においても親よりスコアが悪化する場合, その rule はあらゆる面で親より質の悪い rule という事になるため, そのような rule を出力する意味はないと考える.

2.4 カウント射影クエリ (Count Projection Queries)

AMIE アルゴリズムでは rule の Refinement のために 3 つの演算が行われる. このような演算を KB 上でどのように実装するか説明する.

初めに, head-coverage の閾値を上回る新しい rule が導入できる relation を探索する. これは以下の**カウント射影クエリ** (count projection query) で実現される.

```
SELECT  $r$ , COUNT( $H$ )
WHERE  $H \wedge B_1 \wedge \dots \wedge B_{n-1} \wedge r(X, Y)$ 
SUCH THAT COUNT( $H$ )  $\geq k$ 
```

ここで $k := minHC \times size(H)$ であり, head-coverage の閾値を support による絶対値の閾値へと変換している. COUNT($\cdot$) は COUNT(DISTINCT $\cdot$) のセマンティクスを持つ. X, Y は変数を表し, 新規の変数あるいは rule に既に存在する変数となる. r に対するクエリ結果として, $B_1 \wedge \dots \wedge B_{n-1} \wedge r(X, Y) \Rightarrow H$ の head-coverage が $minHC$ 以上となる relation が得られる. また, 結果として得られる各 relation r に対して, COUNT(H) は新しい rule の support となる.

ここからは Refinement に関する 3 つの演算に対して, 上記の count projection query で実装することを考える.

■Dangling Atom Operator Algorithm 1で以下の non-closed な rule が dequeue されたことを考える.

$$marriedTo(x, z) \Rightarrow livesIn(x, y)$$

この rule に $\mathcal{O}_D$ を適用する count projection query は以下のようになる.

```

SELECT  $r$ , COUNT( $livesIn(x, y)$ )
WHERE  $livesIn(x, y) \wedge marriedTo(x, z) \wedge r(X, Y)$ 
SUCH THAT COUNT( $livesIn(x, y)$ )  $\geq k$ 

```

with $r(X, Y) \in \{r(y, w), r(z, w), r(w, y), r(w, z)\}$. $r(X, Y)$ は新しい dangling atom として rule に結合される可能性のある形式の atom に結び付いており, w は任意の新規となる変数を表す. 中間的な rule に対して, dangling atom は non-closed (open) な変数 (ここでは z と y) に結合される一方, rule が closed であれば, rule に現れる全ての変数と結合される.

■Closed Atom Operator $\mathcal{O}_C$ の処理についても同様に動作する. dangling atom で用いた例に対して, ここでは $r(X, Y)$ が $\{r(z, y), r(y, z)\}$ となり, 全ての open な変数が closed となるような新しい atom を導入する. もし open な変数が 1 つだけの場合, その open な変数と他の全ての変数を引数とする新しい atom が導入される. もし rule が既に closed である場合, rule に存在する全ての変数のペアに対して処理が行われる.

■Instantiated Atom Operator $\mathcal{O}_I$ の処理は 2 つのステップに分けて実装される. 第一のステップでは $\mathcal{O}_D$ の処理を行い, 新しい dangling atom と新規の変数を持った中間的な rule の集合を生成する. 第二のステップでは, 生成された各 rule に対して, 新規の変数上で count projection query を行う. 例えば, $marriedTo(x, z) \Rightarrow livesIn(x, y)$ を例にとると, 第一のステップでは可能な全ての dangling atom を rule に追加する. ここでは $r(x, w)$ の形式の dangling atom が追加されたと考え, $minHC$ 以上の head-coverage を持つ r に対して, 第二のステップでは w に対する最良の代入候補を探索する. 例えば, $r = citizenOf$ とすると, これは次のクエリを考えることに相当する.

```

SELECT  $w$ , COUNT( $livesIn(x, y)$ )
WHERE  $livesIn(x, y) \wedge marriedTo(x, z) \wedge citizenOf(x, w)$ 
SUCH THAT COUNT( $livesIn(x, y)$ )  $\geq k$ 

```

各 w への代入ごとに新しい rule を形成し, それが enqueue された後, rule として出力されるか評価されることになる.

3 クエリ実装の詳細

ここでは節2.4のクエリ処理を実装する詳細について説明する^{*2}.

3.1 インメモリデータベース (In-Memory Database)

節2.4のクエリ処理に特化したインメモリデータベース (In-Memory Database) を構築する. 実装では列 S , R , O に対する順列のパターン 1 つに対して index を用意し, fact を indexing する. すなわち, 6 つの index, SRO , SOR , RSO , ROS , OSR , ORS があり, これらを **fact index** と呼ぶ. 各 fact index はハッシュテーブル (hash table) で実装される. 例えば, ORS の fact index では各 object を key として hash table にマッピングする. さらに, この hash table は relation を key とし, $r(s_i, o) \in \mathcal{K}$ for $i = 1 \dots n$ となるような

^{*2} 文章による記述は抽象度が高いため, 深い内容に興味のある方はアルゴリズムや実装 <https://github.com/lajus/amie> を自力で読み解くことを推奨します.

Algorithm 3 存在クエリ

```
1: function EXISTS( $B_1 \wedge \dots \wedge B_n, \mathcal{K}$ )
2:    $q := B_1 \wedge \dots \wedge B_n$ 
3:   if  $n = 1$  then
4:     return  $\text{size}(B_1, \mathcal{K}) > 0$ 
5:    $s := \underset{i}{\text{argmin}}\{\text{size}(B_i, \mathcal{K})\}$ 
6:    $q := q \setminus \{B_s\}$ 
7:   for all instantiations  $b_s \in B_s$  do
8:      $q' := q$  instantiated with bindings from  $b_s$ 
9:     if EXISTS( $q', \mathcal{K}$ ) then
10:      return true
11:   return false
```

subject の集合 $\{s_1, \dots, s_n\}$ へのマッピングを持つ. fact index は triple の存在を定数時間でチェックするのに役立つ.

fact index に加えて, **aggregated index** と呼ばれる 3 つの index S, P, O も用意する. これらの index は各 key に対する fact の合計数を管理する. 例えば, aggregated index P は各 relation に対する triple の数を管理する. 同様に, S は subject に現れる各 entity に対する triple の数を管理する.

3.2 サイズクエリ (Size Queries)

fact index と aggregated index を利用することで, atom a の size ($\text{size}(a, \mathcal{K})$) を決定することができる. 例えば, $\text{livesIn}(x, y)$ (x と y は変数) の size は aggregated index P を単に調べるだけで決定できる. また, $\text{livesIn}(x, USA)$ に対する size は fact index ROS に対して, まず, livesIn を key として object を検索し, その結果に対して USA を key として subject のリストを得ることで決定できる.

3.3 存在クエリ (Existence Queries)

In-Memory Database の中心的な役割は連言クエリの binding が存在するかどうかを決定することである. Algorithm 3 はこれを実装したものである. 入力として連言クエリと KG $\mathcal{K}$ を受け取り, 3-4 行目ではもしクエリが単一の atom であれば, その size が 0 より大きいかを直接判定する. それ以外は 5 行目において, index を利用し, instantiation の数が最小となる atom B_s を選択する. 7-8 行目では B_s に対する各 instantiation を残りの atom に適用する. 9 行目では上記のプロセスを単一の atom になるまで再起的に繰り返す.

3.4 選択クエリ (Select Queries)

Algorithm 4 は SELECT DISTINCT クエリを記述している. 3 行目では instantiation の数が最小となる atom B_s を選択し, B_s に projection 対象の変数 x が含まれる場合 (5-9 行目), アルゴリズムは x の全ての instantiation $\hat{x}$ に対して, 連言クエリへの適用を実行する. 8-9 行目ではこの適用結果が KG $\mathcal{K}$ に存在するか確認し, $\hat{x}$ を結果として保持する. 逆に B_s に x が存在しなければ (11-15 行目), B_s に対する全ての

Algorithm 4 選択クエリ

```
1: function SELECT( $x, B_1 \wedge \dots \wedge B_n, \mathcal{K}$ )
2:    $q := B_1 \wedge \dots \wedge B_n$ 
3:    $s := \underset{i}{\operatorname{argmin}} \{ \operatorname{size}(B_i, \mathcal{K}) \}$ 
4:    $result = \{ \}$ 
5:   if  $x \in B_s$  then
6:     for all instantiations  $\hat{x} \in x$  do
7:        $q' := q$  instantiated with  $\hat{x}$  for  $x$ 
8:       if Exists( $q', \mathcal{K}$ ) then
9:          $result.add(\hat{x})$ 
10:  else
11:     $q := q \setminus \{B_s\}$ 
12:    for all instantiations  $b_s \in B_s$  do
13:       $q' := q$  instantiated with bindings from  $b_s$ 
14:      if Exists( $q', \mathcal{K}$ ) then
15:         $result.add(\operatorname{SELECT}(x, q', \mathcal{K}))$ 
16:  return  $false$ 
```

instantiation を残りの (B_s を除いた) 連言クエリに適用し、アルゴリズムを再帰的に動作させる。

3.5 カウントクエリ (Count Queries)

rule $\vec{B} \Rightarrow r(x, y)$ の confidence を計算するため、AMIE アルゴリズムではカウントクエリ (count query) を使い、confidence 計算式の分母を推定する。PCA confidence では次のような形式のクエリを考える。

SELECT COUNT(x, y) WHERE $r(x, y') \wedge \vec{B}$

ここで x, y は rule の head の変数、 $\vec{B} = B_1 \wedge \dots \wedge B_n$ は rule の body、 $r(x, y')$ は head の変形であり、object が新しい変数 y' で置換されている。このようなクエリはパターン $r(x, y') \wedge \vec{B}$ を満たす head の変数に対する binding の異なり数を返す。In-Memory Database ではまず以下の SELECT クエリ

SELECT DISTINCT x WHERE $r(x, y') \wedge \vec{B}$

を使って、 x に対する binding を取得し、各 binding に対して、 y に関する別のクエリを使うことで計算を実現する。

3.6 カウント射影クエリ (Count Projection Queries)

count projection クエリは次のような形式で表される。

SELECT x , COUNT(H) WHERE $H \wedge B_1 \wedge \dots \wedge B_n$
SUCH THAT COUNT(H) $\geq k$

Algorithm 5 カウント射影クエリ

```
1: function COUNTPROJECT( $\mathbf{x}$ ,  $R(X, Y) \wedge B_1 \wedge \dots \wedge B_n$ ,  $k$ ,  $\mathcal{K}$ )
2:    $map := \{\}$ 
3:    $q := B_1 \wedge \dots \wedge B_n$ 
4:   if  $\mathbf{x} \in \{R, X, Y\}$  then
5:     for all instantiations  $r(x, y) \in R(X, Y)$  do
6:        $q' := q$ , replace  $R$  by  $r$ ,  $X$  by  $x$ ,  $Y$  by  $y$ 
7:       if Exists( $q', \mathcal{K}$ ) then
8:          $map[x] = map[x] + 1$ 
9:   else
10:    for all instantiations  $r(x, y) \in R(X, Y)$  do
11:       $q' := q$ , replace  $R$  by  $r$ ,  $X$  by  $x$ ,  $Y$  by  $y$ 
12:       $\mathcal{X} := \text{SELECT}(\mathbf{x}, q', \mathcal{K})$ 
13:      for all  $x \in \mathcal{X}$  do
14:         $map[x] = map[x] + 1$ 
15:    $map := \{\langle x \rightarrow n \rangle \in map : n \geq k\}$ 
16:   return  $map$ 
```

これは AMIE アルゴリズムの Refinement 処理の段階で新しく追加された atom に対して、その relation や instance を決定するために使われる。Algorithm 5はこのクエリへの応答をどのように行うかを表しており、入力として selection 対象の変数 $\mathbf{x}$ 、projection 対象の atom $H := R(X, Y)$ 、残りの atom $B_1 \wedge \dots \wedge B_n$ 、閾値 k 、KG $\mathcal{K}$ を受け取る。アルゴリズムは key として x ($\mathbf{x}$ への値) に対する instantiation、value として H に対する binding の異なり数を持った hashtable を返す。

3 行目では $\mathbf{x}$ が projection 対象の atom に出現するかをチェックする。もし出現する場合 (5-8 行目), projection 対象となる atom に対する全ての instantiation を処理し、それをクエリに適用する。そしてクエリが KG に存在する場合、selection 対象となる変数 $\mathbf{x}$ に対応する各値 x のカウンタを増加させる。次に $\mathbf{x}$ が projection 対象となる atom に存在しない場合 (10-14 行目), projection 対象となる atom に対する全ての instantiation を処理し、それをクエリに適用する。12 行目で、 $\mathbf{x}$ に対して、SELECT DISTINCT クエリを実行し、その各結果 x に対して、カウンタを増加させる。15 行目では閾値 k を上回る結果を map に残す。

4 Horn 規則マイニングにおける指標の改善

表2 KG の例 (表形式): 3つの relation *livesIn*, *isPoliticianOf*, *diedIn*, 及び、9つの fact から構成される。

<i>livesIn</i>	<i>isPoliticianOf</i>	<i>diedIn</i>
(Ava, Paris)	(Ava, Paris)	(Ava, Paris)
(Emily, London)	(Bob, Newyork)	(Bob, Newyork)
(Emily, Paris)	(Ava, Newyork)	(Emma, Lisbon)

ここでは PCA の問題点について考え、それを解決する指標を紹介する [4]。PCA は *diedIn* のような1つの

subject がただか 1 つの object を持つような relation には適した指標であるが、各 subject が複数の object を持つような relation では問題が起こる。例えば、表2の KB と下記の rule R

$$livesIn(x, y) \Rightarrow isPoliticianOf(x, y)$$

を考える。このとき、 R からは予測として

$$\begin{aligned} &isPoliticianOf(Ava, Paris) \\ &isPoliticianOf(Emily, London) \\ &isPoliticianOf(Emily, Paris) \end{aligned}$$

の 3 つの fact が生成され、この結果から支持度は 1 であることが分かる。また、PCA を計算すると、 $conf_{pca}(R) = \frac{1}{1+0} = 1$ となる。一般に、ある都市の住民が必ず政治家であるということは考えられないので、rule R の確信度を 1 として推定することは明らかに問題がある。以下では、文献 [4] において考案された改善案を紹介する。

ここで rule $R = B \Rightarrow r(x, y)$ として、rule R によって予測される事実の集合を $P_R = \{p \mid (K \wedge R \models p)\}$, 其中で KG K に存在する事実の集合を $KP_R = \{p \mid (K \wedge R \models p) \wedge p \in K\}$ と定義する。また、rule R によって予測される事実で K に存在しない事実の集合を U_R と定義する。 U_R はさらに真となる事実の集合 UP_R と負例となる事実の集合 UN_R に分けられる。これを以下の図3に示す。このような整理に基づくと、本来、

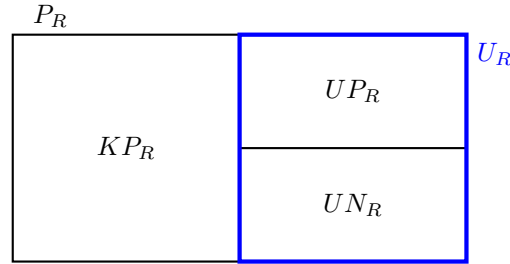


図3 rule R で予測された事実の区分 (文献 [4] から引用).

計算したい確信度は

$$\frac{support(R) + |UP_R|}{|P_R|}$$

と定義できる。そして、これを計算するには $|UP_R|$, すなわち、 R で予測された事実の中で真となる事実の集合の大きさが分かれば良いことになる。

文献 [4] では $|UP_R|$ を計算するため、様々な工夫が考案されている。まず、ヘッド被覆率に対する以下のような仮定を考案している (**被覆率に関する関係性**)。 UP_r は KG K では未登録となっているが、relation r によって作られる真の事実の集合を表す。

$$rc(R) = \frac{support(R)}{|(x, y) : r(x, y) \in K|} = \frac{|UP_r|}{|UP_r|}.$$

$rc(R)$ は rule R によって予測される事実に対して、既知の領域における真の事実が予測される割合と未知の領域における真の事実が予測される割合が同じになる、という仮定である。 $rc(R)$ の計算は自明であるため、もし $|UP_r|$ を計算することができれば、この仮定の下では $|UP_R|$ を計算することができる。

4.1 被覆率仮説 (Coverage Assumption)

前節では, rule $R = \vec{B} \Rightarrow r(x, y)$ に対して,

$$rc(R) = \frac{\text{support}(R)}{|(x, y) : r(x, y) \in K|} = \frac{|UP_R|}{|UP_r|}$$

という仮定を導入した. これは rule が既知領域において真の事実を被覆する割合と, 未知領域において真の事実を被覆する割合が等しいという仮定である.

この仮定に基づくと,

$$|UP_R| = rc(R) \times |UP_r|$$

が成り立つ. したがって, 未知領域に存在する真の事実数 $|UP_r|$ を推定することができれば, rule R によって予測される未知の真事実数 $|UP_R|$ を推定できることになる.

4.2 未知の真事実数の推定 (Estimating the Number of Unknown Positive Facts)

relation r に対して,

$$\beta = \frac{|UP_r|}{|U_r|}$$

を未知事例のうち真事例が占める割合と定義する. ここで U_r は relation r に関する未知事例の集合である.

もし β が既知であれば,

$$|UP_r| = \beta |U_r|$$

によって未知の真事例数を求めることができる. しかし実際には β は未知であるため, 何らかの方法で推定する必要がある.

文献 [4] では relation ごとに一律の値を用いるのではなく, 各 rule に対して個別に β を推定する方法を提案している. rule $R = \vec{B} \Rightarrow r(x, y)$ に対して,

$$S_R = \{x \mid \exists y : r(x, y) \in P_R\}$$

を予測結果に現れる subject の集合,

$$O_R = \{y \mid \exists x : r(x, y) \in P_R\}$$

を予測結果に現れる object の集合とする. このとき,

$$\beta_R = \frac{\text{support}(R)}{|S_R| \times |O_R|}$$

を rule R に対する未知事例の正例率とする.

4.3 未知事例の分類 (Partitioning Unlabeled Predictions)

rule R による未知事例集合 U_R を考える. ここで,

$$S_r = \{x \mid \exists y : r(x, y) \in K\}$$

を relation r において subject として観測された entity の集合,

$$O_r = \{y \mid \exists x : r(x, y) \in K\}$$

を relation r において object として観測された entity の集合とする.

このとき, 未知事例集合 U_R を以下の 4 つの部分集合に分割する.

$$U_R = U_{SoldOold} \cup U_{SoldOnew} \cup U_{SnewOold} \cup U_{SnewOnew}.$$

ここで,

$$U_{SoldOold} = \{r(x, y) \mid x \in S_R \cap S_r, y \in O_R \cap O_r\} \setminus K$$

であり, subject と object の両方が既知である予測を表す. 同様に,

$$U_{SoldOnew} = \{r(x, y) \mid x \in S_R \cap S_r, y \in O_R \setminus O_r\},$$

$$U_{SnewOold} = \{r(x, y) \mid x \in S_R \setminus S_r, y \in O_R \cap O_r\},$$

$$U_{SnewOnew} = \{r(x, y) \mid x \in S_R \setminus S_r, y \in O_R \setminus O_r\}$$

と定義する. PCA と同様に, $U_{SoldOold}$ に属する予測は負例として扱う. これは subject と object が既に relation r に出現しているにも関わらず, その組が KG に存在しないためである.

4.4 関数性による補正 (Functionality-Based Adjustment)

未知事例数を推定する際には relation の関数性を考慮する必要がある. 例えば, 出生地や死亡地のような relation では, ある subject が新しい object と結び付く可能性は低い. 一方で, 居住地のような relation では複数の object を持つことが自然である.

この違いを考慮するため, 文献 [4] では次の補正係数を導入している.

$$f_S(r) = 1 - HM_{x \in S_r} \left(\frac{1}{|\{y : r(x, y) \in K\}|} \right)$$

および

$$f_O(r) = 1 - HM_{y \in O_r} \left(\frac{1}{|\{x : r(x, y) \in K\}|} \right)$$

である. ここで HM は調和平均 (harmonic mean) を表す. relation が function に近い場合には $f_S(r)$ は小さくなり, inverse function に近い場合には $f_O(r)$ は小さくなる.

4.5 RC Confidence

以上の定義を用いると, 各部分集合に含まれる未知の正例数を

$$|UP_{SoldOnew}| = |U_{SoldOnew}| \times \beta_R \times f_S(r) \times rc(R),$$

$$|UP_{SnewOold}| = |U_{SnewOold}| \times \beta_R \times f_O(r) \times rc(R),$$

$$|UP_{SnewOnew}| = |U_{SnewOnew}| \times \beta_R \times rc(R)$$

として推定することができる. したがって,

$$|UP_R| = |UP_{SoldOnew}| + |UP_{SnewOold}| + |UP_{SnewOnew}|$$

となる。最終的に Coverage Assumption に基づく confidence は

$$conf_{rc}(R) = \frac{\text{support}(R) + |UP_R|}{|P_R|}$$

として定義される。

PCA confidence は relation が function に近い場合には有効であるが、複数の object を持つ relation に対しては過大評価を行うことがある。一方、RC confidence は未知領域に存在する正例数を推定することで、そのような relation に対してもより妥当な評価を与えることができる。

参考文献

- [1] Luis Galárraga, Christina Teflioudi, Katja Hose, and Fabian M Suchanek. Fast rule mining in ontological knowledge bases with amie+. *The VLDB Journal*, 24(6):707–730, 2015.
- [2] Luis Antonio Galárraga, Christina Teflioudi, Katja Hose, and Fabian Suchanek. Amie: association rule mining under incomplete evidence in ontological knowledge bases. In *Proceedings of the 22nd international conference on World Wide Web*, pages 413–422, 2013.
- [3] Maximilian Nickel, Kevin Murphy, Volker Tresp, and Evgeniy Gabrilovich. A review of relational machine learning for knowledge graphs. *Proceedings of the IEEE*, 104(1):11–33, 2016.
- [4] Kaja Zupanc and Jesse Davis. Estimating rule quality for knowledge base completion with the relationship between coverage assumption. In *Proceedings of the 2018 World Wide Web Conference*, pages 1073–1081, 2018.